

Contents

1	How to use these notes	3
2	Why keystrokes?	3
	Title slide	3
	Every login factor has a weak spot	4
	The one-time gate problem	5
	Same word, different hands	5
	Where keystroke dynamics stands out	5
	KeyLoggd at a glance	6
	Gamified enrollment: the test is the sensor	6
	Gamified identification: type until recognised	7
	Limitations, honestly	7
3	Under the hood	8
	Architecture: two halves that never mix	8
	End-to-end data flow	8
	Step 1 · Capture: timestamping every key	8
	Step 2 · Dwell, flight and rollover	9
	Step 3 · Slicing one long run into samples	9
	A real sample, as the pipeline sees it	9
	What our six real typists look like	10
	Step 4a · The frequency-domain view of rhythm	10
	Step 4b · Denoising with an FFT low-pass filter	11
	Step 4c · Why the spectrum lost	11
	Step 5a · Taming pauses and rollover with asinh	12
	Step 5b · The 26-dimensional feature vector	12
	Block 1 · Robust statistics of each timing stream	12
	Blocks 2 & 3 · Key habits and cross-stream ratios	13
	Step 6 · Normalise: one real vector as z-scores	13
	Step 7 · Templates: a point plus a spread	14
	Step 8 · Distance: scaled Manhattan	14
	Step 9 · Identification: distance-weighted k-NN	15
	Step 10 · Verification: genuine vs impostor	15
	Step 11 · Equal error rate and operating point	15
	Step 12 · Fusing a whole run into one verdict	16
	The decision, end to end	16

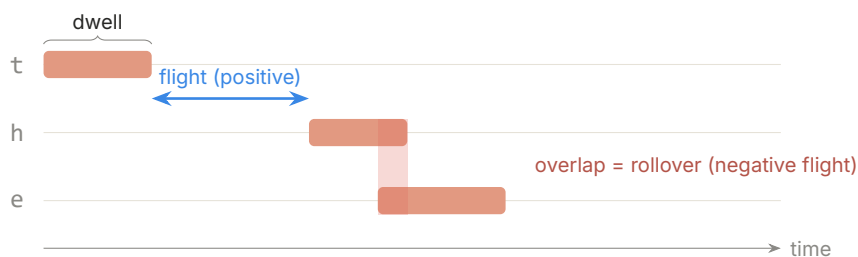
A real identification, bar by bar	17
How we measured it	17
Results	17
Engineering the app	18
Explain mode: the pipeline, one phase at a time	18
Running it	18
Takeaways & what's next	19
4 Numbers to remember	19
5 Questions you are likely to get	20

1 How to use these notes

The notes follow the slides in order. Each slide title appears as a heading, followed by the words on it that need explaining. For each word you get:

- a **plain explanation**: what it is, without jargon;
- where it helps, **why we use it** in KeyLoggd;
- for the important ones, a 🗣️ *On stage* line you can say almost word for word.

The whole project in three sentences. Everyone presses and releases keys with a personal rhythm. KeyLoggd records that rhythm while you play a typing game, turns it into 26 numbers that describe *how* you type, and stores an average profile for you. Later, when someone types, it compares their numbers to every stored profile and names the closest one, or says “unrecognised” if nobody is close enough.



The only two measurements everything is built from: how long each key is held (**dwell**), and the gap until the next key goes down (**flight**).

2 Why keystrokes?

🖥️ Title slide

Keystroke dynamics

The study of *how* someone types, not *what* they type: the timing of each key press and release. It is a kind of behavioural biometric.

Behavioural biometric

A way of recognising a person from something they *do* (typing, walking, signing, moving a mouse), as opposed to a *physiological* biometric, which uses something they physically *are* (fingerprint, face, iris).

Rhythm

Our informal word for the pattern of holds and gaps as you type. The bars on the title slide are a real 32-key sample from our data: orange = dwell, blue = flight.

Think-pause

A long gap while the typist stops to think or read (1.37 s in the title-slide sample). It is about 20 times a normal gap, which is why the pipeline has to be careful with averages.

📺 Every login factor has a weak spot

Authentication

Proving you are who you claim to be. Different from *authorisation*, which decides what you are allowed to do once you are in.

Authentication factor

A category of proof. The classic three are something you *know* (password), something you *have* (phone, token), and something you *are* (fingerprint). Keystroke dynamics adds a fourth: something you *do*.

PIN / pattern

Short secrets, a number or a swipe shape. Same weaknesses as passwords, just shorter.

Phishing

Tricking someone into typing their secret into a fake site or form.

Credential reuse

Using the same password on many sites, so one leak unlocks all of them.

Shoulder-surfing

Watching someone type their password or PIN.

Reset flow

The “forgot password” process. It is often the weakest link, because it has to let you in without the password.

OTP (one-time password)

A short code that works once, usually sent by SMS or generated by an app.

2FA (two-factor authentication)

Requiring two different factors, typically password plus OTP.

SIM swap

An attacker convinces a mobile carrier to move your number to their SIM card, so your SMS codes go to them.

Real-time phishing

A fake site that forwards your OTP to the real site instantly, before it expires.

Sensor

Dedicated hardware to read a biometric (fingerprint reader, depth camera). Keystroke dynamics needs none: the keyboard is the sensor.

Revocable / irrevocable

Whether you can replace the secret after it leaks. You can change a password, but not your fingerprint.

Spoofing

Faking a biometric: a printed photo for face unlock, a silicone mould of a fingerprint.

🎤 *On stage:* “Passwords get phished, phones get SIM-swapped, fingerprints need a sensor and can’t be

changed. And all three only check you once, at the door."

🖥 The one-time gate problem

Session

The period after you log in, during which the system trusts you without asking again.

Session takeover

Someone else using your already-open session, for example on a laptop you left unlocked.

Continuous authentication

Checking identity again and again during the session, quietly, instead of only at login. Typing rhythm is a natural fit because people type all the time.

Re-authenticate

Asking the user to prove identity again (for example, retype the password) when something looks wrong.

🗣 *On stage:* "A password is a door check. Your typing never stops being you, so keystroke dynamics can become a continuous signal."

Be precise if asked: KeyLoggd today identifies someone from a typing test. Continuous background checking is where the idea leads (see "What's next"), not something the app does yet.

🖥 Same word, different hands

Dwell time

How long one key is held down: release time minus press time. Typically 50–150 ms.

Flight time

The gap between releasing one key and pressing the next: next press minus this release.

Rollover

Pressing the next key *before* releasing the previous one. The flight time becomes *negative*. Fast typists do it constantly; in our data ahnaf rolls over on 36 % of keys and arghya never does.

Median gap

The middle value of all a person's flight times. ahnaf's is 2 ms (almost every key overlaps), arghya's is 103 ms.

🖥 Where keystroke dynamics stands out

The scorecard

Each method is rated strong (✅), partial (⚡) or weak (❌) on seven criteria. The "Overall" bar counts strong as 1 and partial as ½, out of 7. This is *our qualitative judgement*, not a measurement: say so if asked.

"Can't be copied just by watching"

You can see a password being typed; you cannot see someone's millisecond timing.

Accuracy today (partial)

Keystroke dynamics is less exact than a password: it gives a similarity score, not a yes/no match. That is why we position it as a *second* signal.

 *On stage: "Keystrokes win on cost, effort and continuity, and lose on raw accuracy. That's why it's a strong second signal, not a replacement."*

KeyLoggd at a glance

Enroll

Registering a new user: they type, we store samples of their rhythm under their name.

Identify

Given new typing, decide which enrolled person it is, or nobody.

Explain

A mode that walks through the pipeline step by step on the user's own keystrokes, with the real numbers.

Sample

One chunk of 32 consecutive keystrokes. It is the unit everything is measured on.

Template

The stored profile of one user: the average of their samples plus how much they vary.

Verdict

The final answer: a name, or UNRECOGNIZED.

UNRECOGNIZED

The answer when even the closest profile is too far away. This is what makes the system *open-set*.

"0 ML libraries, plain numpy"

We did not use scikit-learn or any machine-learning package. Everything (scaling, nearest neighbours, templates, error rates) is written by hand on top of **numpy**, Python's standard array/maths library.

"100 % benchmark accuracy, 5 samples fused"

On our synthetic benchmark, combining 5 samples from one run into a single decision identified every typist correctly.

Gamified enrollment: the test is the sensor

Gamification

Using game elements (scores, timers, progress, rewards) in something that is not a game, here to collect good data.

Monkeytype

A popular web typing test. Our test copies its look and behaviour.

Modes

time (type for 15–120 s), *words* (type 10–100 words), *quote* (a fixed passage), *zen* (free typing until Enter), *custom* (your own text). Punctuation and numbers can be switched on.

Caret

The text cursor. Ours glides smoothly between letters.

Pace caret

A ghost cursor that moves at a target speed (40–300 wpm) so you can race it.

wpm (words per minute)

Characters typed divided by 5 (a “standard word”), per minute. **Net wpm** counts only correct characters; **raw wpm** counts everything typed in that second.

Accuracy

Correct characters as a share of all characters typed.

Enrollment milestone

A pop-up at 12 samples: enough for a stable template.

 *On stage:* “People chasing a score type naturally. One 30-second test gives us four or five samples at once.”

Gamified identification: type until recognised

Threshold (τ , tau)

The maximum distance at which we still accept a match. Closer than τ : we name the person. Farther: UNRECOGNIZED.

$d_{\min}$

The distance to the *closest* enrolled profile.

Retry loop

If nobody passes the threshold, a new test is dealt instantly; more typing means more evidence.

Margin

How much closer the winner is than the runner-up. A big margin means a confident answer.

Limitations, honestly

EER (equal error rate)

The single-number error summary for biometrics; explained fully under Step 11. Lower is better. Ours is 19.0 % on real data and 5.9 % on the synthetic benchmark.

One sitting / cross-session

All real samples were typed in one session per person. We have not yet tested whether it still works on a different day, when people type differently.

Normalisation shift

Features are scaled using statistics of the whole enrolled group, so adding a new user slightly changes everyone’s scaled numbers.

Liveness check

A test that the input comes from a live human right now, not a recording.

Replay attack

Recording someone’s keystroke timings and playing them back to the system.

Trained mimic

A person who practises imitating another person’s rhythm.

Impostor accept

Letting the wrong person in (a false accept).

Handler bias / jitter

Small timing errors added by the software itself: about 5 ms constant delay, plus about 3 ms of random wobble when animations are running. Human dwell times vary by 10–30 ms, so this is small.

3 Under the hood

🖥 Architecture: two halves that never mix

Tkinter (Tk)

Python's built-in toolkit for desktop windows and buttons. Our whole interface is drawn with it.

Standard library

The modules that ship with Python itself; no installation needed.

numpy

The standard Python library for fast number arrays and maths. Our only third-party dependency.

Pipeline

The chain of steps that turns raw key events into a verdict. It lives in `keyloggd/pipeline` and never touches the UI.

Module names

`samples` (cut runs into 32-key windows), `signal_construction` (dwell and flight), `timing_features` (the 26 numbers), `fft_features` (the spectrum), `denoise` (low-pass filter), `classifier` (templates, k-NN, EER), `identify_sample` (fusion and verdict).

tools/

`synthetic_data_generator` creates fake typists; `benchmark_inference` measures accuracy.

JSON

A plain-text data format. Each enrolled user is one `.json` file in `data/synthetic/`.

🗣 *On stage:* "The maths never imports the UI, and the UI never does maths. Either half can be tested alone."

🖥 End-to-end data flow

The twelve boxes

Read them as a story: capture key events → timestamp them → cut them into samples → turn each sample into dwell/flight signals → compute 26 features → scale them → build user templates → measure distances → vote with neighbours → combine the run → compare with the threshold → verdict. Colours: orange capture, blue signal and features, amber model, green decision.

☐ Step 1 · Capture: timestamping every key

KeyPress / KeyRelease

The two events Tk reports for every key: when it goes down and when it comes up.

time.perf_counter()

Python's highest-resolution clock. It only moves forward, so it is ideal for measuring short intervals.

Microsecond (μ s)

One millionth of a second. We round timestamps to 6 decimal places, i.e. to the microsecond.

Auto-repeat

When you hold a key, the operating system sends repeated "press" events. We ignore them with a **held-key set** (a list of keys currently down), otherwise we would pair the wrong press and release.

Schema

The agreed structure of a data file: user id, phrase, and a list of samples, each a list of {key, type, t} events.

☐ Step 2 · Dwell, flight and rollover

$d[n]$ and $f[n]$

The dwell and flight of keystroke number n . The square brackets are the usual notation for a *discrete* signal: a list of values indexed by a whole number.

$t_n^\downarrow, t_n^\uparrow$

Press time and release time of key n .

N keys $\Rightarrow N - 1$ flights

There is no gap after the last key.

Discrete-time signal

A sequence of numbers, one per step. Here a "step" is one keystroke, not a fixed slice of time.

Impulse train, $\delta(t - t_i)$

A mathematical way to describe "something happened at these instants": a spike at each key press. We do not use it in code; it justifies treating keystrokes as a signal.

☐ Step 3 · Slicing one long run into samples

Pairing

Matching each press to the next release of the same key, which works even when keys overlap.

Window of 32

We cut the run into consecutive 32-key chunks. Long enough for a stable median, short enough to get several per test.

MIN_KEYS = 16

A leftover chunk shorter than 16 keys is thrown away: too few keys to describe the typist.

Rebasing

Shifting each chunk's timestamps so it starts at $t = 0$. Harmless, since only differences are used.

☞ A real sample, as the pipeline sees it

The chart

Orange bars = dwell of each key, blue bars = flight after it. Blue bars below zero are rollover. The letters under the axis are what was typed (dots are spaces), typos included.

Clipped

The 1370 ms pause is far off the top of the chart, so the bar is cut and labelled.

🗣️ *On stage: "The features never read the text. Only timing. That's why typos and free text don't matter."*

☞ What our six real typists look like

$\tilde{d}$, $\tilde{f}$ (tilde)

The tilde means *median*. $\tilde{d}$ = median dwell, $\tilde{f}$ = median flight.

Median

The middle value when sorted. Unlike the average, one huge pause cannot drag it.

Bubble chart

Each bubble is a person: left-right is median flight, up-down is median dwell, bubble size is rollover rate. People far apart on this chart are easy to tell apart.

☞ Step 4a · The frequency-domain view of rhythm

Time domain vs frequency domain

Two views of the same signal. Time domain: the values one after another. Frequency domain: how much of the signal is slow, smooth variation versus fast back-and-forth alternation.

DFT / FFT

The Discrete Fourier Transform rewrites a list of N numbers as a sum of waves of different frequencies. The FFT (Fast Fourier Transform) is simply the fast algorithm that computes it.

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}$$

$X[k]$ says how strongly a wave that repeats k times across the window is present.

Frequency bin k

One of the output slots of the FFT. Bin 0 is the average level (called **DC**); bin 1 is one slow cycle across the sample; bin 16 is the fastest possible alternation (long, short, long, short...).

$|X[k]|$ (magnitude)

The strength of bin k , ignoring its timing (phase).

$N = 32$

The FFT length. Every sample is brought to exactly 32 values first.

Zero-padding

Adding zeros to the end to reach length 32. Fine for a fixed phrase, but it makes the spectrum depend on text length.

Resampling / linear interpolation

Stretching or squeezing the sequence onto exactly 32 points by drawing straight lines between the original values. Keeps the shape regardless of length.

Conjugate symmetry

For a real-valued input, the second half of the spectrum mirrors the first. So 32 inputs give only 17 unique bins (0 to 16).

Spectral centroid

The “centre of mass” of the spectrum: the average bin, weighted by strength. High centroid = jerky rhythm.

Band energy ratio

The share of total energy in the low, mid and high bins.

Total energy

The sum of squared magnitudes. Dominated by any big pause, which is its weakness.

The original 14-D vector

5 spectral numbers for dwell + 5 for flight + mean and standard deviation of each. “14-D” = 14 dimensions (numbers).

🖥 Step 4b · Denoising with an FFT low-pass filter

Low-pass filter

Keeps slow variation and removes fast wiggles. Here: take the FFT, set the high bins to zero, transform back.

IFFT

Inverse FFT: turns a spectrum back into the original kind of signal.

Cutoff $c = 5$

We keep bins 0–4 (and their mirror images 27–31) and zero everything in between: the lowest 35 % of the spectrum.

Jitter

Small random run-to-run timing noise. It lives mostly in the high bins.

$\text{Re}\{\cdot\}$

“Real part”. Zeroing symmetrically keeps the result real; we drop the tiny imaginary rounding error.

🖥 Step 4c · Why the spectrum lost

Estimation noise

With only about 30 keys, 17 frequency values are mostly noise, not structure.

Length leakage

Zero-padding makes the spectrum change with how long the text was, not just who typed it.

Fused

Several samples from one run combined into one decision (Step 12).

🗣 *On stage:* “We started with the spectrum because it’s the clearest picture of rhythm. We measured it, it lost: 68.8 % versus 92.5 %. So it explains, and statistics decide.”

☐ Step 5a · Taming pauses and rollover with asinh

ln (natural logarithm)

Squashes big numbers: it turns "20 times bigger" into "3 more". But it is undefined for zero and negative numbers.

asinh (inverse hyperbolic sine)

$\text{asinh}(x) = \ln(x + \sqrt{x^2 + 1})$. Behaves like a straight line near zero and like a logarithm for large values, and it works for negative numbers too.

$s = 0.05$ s (scale)

Dividing by 50 ms puts ordinary flight times in the straight-line region, so only real pauses get squashed.

Sign-aware

Keeps negative values negative, so rollover information survives.

Log-ratio, $a(x) - a(y)$

Comparing two values by subtracting their compressed versions. Like a ratio, but it never divides by zero.

☐ Step 5b · The 26-dimensional feature vector

Feature

One number that describes a sample (for example "median dwell").

Feature vector

The list of all 26 features of one sample. Every sample becomes a point in a 26-dimensional space; similar typing = nearby points.

The four blocks

8 dwell statistics + 9 flight statistics (8 plus rollover) + 6 key-class habits + 3 ratios = 26.

Robust statistic

A summary that one extreme value cannot move much (median, IQR, MAD), unlike the mean and standard deviation.

Speed-invariant

Unchanged when the person simply types faster or slower overall; ratios have this property.

NaN / $\pm\infty$

"Not a Number" and infinity: what a division by zero produces. We replace them with 0 so nothing breaks.

☐ Block 1 · Robust statistics of each timing stream

Percentile / quantile ($Q_{10}, Q_{25}, Q_{50}, Q_{75}, Q_{90}$)

Q_{25} is the value below which 25 % of the data lies. Q_{50} is the median.

aMu, aSd

Mean and standard deviation of the asinh-compressed values.

Standard deviation (std)

The usual measure of spread: typical distance from the mean.

iqr (interquartile range)

$Q_{75} - Q_{25}$: the width of the middle half of the data.

mad (median absolute deviation)

The median of $|x - \text{median}|$: a spread measure that ignores outliers.

p90 (10–90 span)

$Q_{90} - Q_{10}$: width of the middle 80 %.

rcv (robust coefficient of variation)

MAD divided by the median: spread relative to the typical value, so it does not depend on speed.

out1 (long-outlier share)

The fraction of keys much slower than usual (above median + 2×MAD): how often the person hesitates.

roll (rollover share)

The fraction of negative flights. Only for flight, since a dwell cannot be negative.

Pr[.]

"Probability of", read here as "fraction of keys for which".

Blocks 2 & 3 · Key habits and cross-stream ratios

Key-class

Grouping keys into spaces and letters. A full per-letter table needs a fixed password; 30 keys of free text touch most letters only once.

spDw, ltDw

Median hold time of the spacebar and of letters. "Punch or brush the spacebar".

aSp, inW

Median gap after a space (starting a new word) and inside a word.

s:l, a:i

Compressed ratios of those two pairs.

d:f (hold vs gap)

How long you hold compared with how long you wait.

duty (duty cycle)

The fraction of each key cycle spent holding the key: $\tilde{d}/(\tilde{d} + |\tilde{f}|)$, always between 0 and 1. The term comes from electronics (how long a signal is "on").

kps (keys per second)

Raw typing speed: number of keys divided by total elapsed time.

ε (epsilon)

A tiny number added to avoid dividing by zero.

Bounded

Guaranteed to stay within a fixed range, so one strange sample cannot produce an enormous value.

🖥 Step 6 · Normalise: one real vector as z-scores

Normalisation / z-score

$z = (x - \mu) / \sigma$: how many standard deviations a value is above (positive) or below (negative) the group average. It puts features with very different units on one common scale.

μ (mu), σ (sigma)

Mean and standard deviation.

Fit on training data only

The averages used for scaling are computed *without* the sample being tested. Otherwise the test would secretly help grade itself; this mistake is called **data leakage**.

Fold

One round of a cross-validation (see Step 9).

Reading the chart

Orange = above the group average, blue = below. The big blue bars show this sample's spacebar is held unusually briefly (55 ms versus 91 ms for letters).

🖥 Step 7 · Templates: a point plus a spread

Template

For each user: the average z-scored vector (μ_u) plus, per feature, how much that user varies (σ_u).

Ellipses in the picture

Solid = one standard deviation, dashed = two. User A is "metronomic" (tiny spread), B is "erratic" (large spread).

Spread floor (SPREAD_FLOOR = 0.25)

A user's spread is never allowed below 25 % of the whole group's spread. With only a few samples, a spread can be near zero by luck, and dividing by it would blow up.

σ_{pop}

The population (everyone's) standard deviation.

🖥 Step 8 · Distance: scaled Manhattan

Distance

A single number for "how different are these two vectors". Small = similar.

Euclidean distance

Straight-line distance: square the differences, add, take the square root. One huge difference dominates.

Manhattan distance (L1)

Add up the absolute differences, like walking city blocks. One bad feature cannot dominate.

Scaled Manhattan

Our version: each feature's difference is divided by *that user's* spread, then averaged over 26 features. A small slip counts a lot for a steady typist and little for an erratic one.

Gaussian (bell curve)

The picture shows two normal distributions. The same deviation of 0.9 is 3 standard deviations for A (reject) but only 0.9 for B (fine).

 *On stage:* "Dividing by each user's own spread was the biggest single improvement: error rate from 7.98 to 5.45 percent."

Step 9 · Identification: distance-weighted k-NN

k-NN (k-nearest neighbours)

Find the k stored samples closest to the new one and let them vote on who it is. We use $k = 3$.

Distance-weighted vote

Each neighbour's vote is worth $1/d$: a very close neighbour counts much more than a far one. It also avoids arbitrary ties.

arg max

"The option that gives the maximum"; here, the user with the highest total vote.

$\mathcal{N}_3(q)$

The set of the 3 nearest neighbours of the query q .

Query

The new, unknown sample we are trying to identify.

LOOCV (leave-one-out cross-validation)

A fair way to measure accuracy on little data: hide one sample, train on all the others, predict the hidden one; repeat for every sample. Result: 38 of 48 correct = 79.2 %.

Chance level

Accuracy of pure guessing: with 6 people, 1 in 6 $\approx$ 17 %.

Step 10 · Verification: genuine vs impostor

Identification vs verification

Identification: "who is this?" (one against many). *Verification:* "is this really Alice?" (one against one, yes or no).

Closed-set vs open-set

Closed-set assumes the typist is definitely one of the enrolled users. Open-set allows "none of them", which is what UNRECOGNIZED gives us.

Genuine score

Distance from a sample to its *own* user's template. Should be small.

Impostor score

Distance from a sample to *someone else's* template. Should be large.

The histogram

Green = genuine scores, red = impostor scores. The overlap between the two humps is exactly where mistakes happen. Real data: 48 genuine and 240 impostor scores; means 1.04 and 2.48.

🖥️ Step 11 · Equal error rate and operating point

FAR (false accept rate)

The share of impostors wrongly accepted at a given threshold. Rises as the threshold loosens.

FRR (false reject rate)

The share of genuine users wrongly rejected. Falls as the threshold loosens.

EER (equal error rate)

The point where FAR = FRR. One number that summarises how well genuine and impostor scores separate, regardless of threshold. Real data: 19.0 %; benchmark: 5.9 %.

Threshold sweep

Trying 500 thresholds between the smallest and largest score to find where the curves cross.

Operating point

The threshold you actually run the system at. It does not have to be the EER point.

HTER (half total error rate)

$(FAR + FRR) / 2$: the average of the two error rates. Minimising it means making the fewest mistakes overall, counting a stranger let in and a real user turned away as equally bad.

ACCEPT_TOLERANCE = 1.05

We run at 1.05× the EER threshold because that minimises HTER on the benchmark (the exact optimum is 1.06×): 5.0 % of genuine users rejected, 7.3 % of impostors accepted. It used to be 1.2, which let more impostors in (11.7 %) to reject fewer real users (2.5 %); that was a convenience choice, not the most accurate one. On real data 1.05 also beats 1.2 (HTER 17.3 % vs 19.0 %).

🗣️ *On stage:* "The threshold decides *whether* we answer, never *who*. The closest profile is always the same."

🖥️ Step 12 · Fusing a whole run into one verdict

Fusion / pooling

Combining evidence from several samples. We average each user's distance across all samples of the run, then pick the smallest average.

Why not a vote?

A vote throws away how close the second-best was. Averaging keeps it, so one bad sample (the "phone buzzed" row) cannot flip the answer.

Mean vs median vs min pooling

Three ways to combine distances; the mean measured best.

Illustrative numbers

The table on this slide is made up to show the idea. The bar chart (92.5 → 97.9 → 100 %) is measured.

🖥️ The decision, end to end

Diamonds

Yes/no decisions in the flowchart.

Confidence levels

From the margin to second place: above 1.0 is high, above 0.4 moderate, otherwise low.

Worker thread

A second line of execution that does the heavy maths so the window never freezes.

Polling a queue

The window checks every 120 ms whether the worker has put a result in a shared "mailbox" (the queue).

🖥 A real identification, bar by bar

In-sample

The sample shown was also part of the enrolled data, so it is easier than a truly new sample. We show it only to illustrate the mechanics; say this if asked.

Reading it

swayam's template is at 0.72, well under the threshold 1.56; the runner-up tahmid is at 1.50, a margin of 0.78, so the verdict is moderate confidence.

🖥 How we measured it

Leave-one-phrase-out

Build templates from 5 of the 6 sentences, test on the sixth, rotate. It checks that we recognise the *typist*, not the sentence.

Leave-one-sample-out

Hide a single sample at a time (the LOOCV above). Used on the real data, where every sample is different free text.

Synthetic fixture

Fake but realistic typists generated by code, so we can test on more data than we collected.

Pangram

A sentence using every letter of the alphabet ("sphinx of black quartz, judge my vow").

$\mathcal{U}(a, b)$

Uniform random value: equally likely anywhere between a and b .

$\text{Exp}(\cdot)$

Exponential distribution: mostly short values with occasional long ones; a natural model for pauses.

Per-key bias

Each simulated user is consistently slower or faster on particular keys, like a real person.

Keyboard realism

We added rollover and pauses to the fake data because the plain generator could never produce them.

🖥 Results

dim

Number of features in the vector (14 spectral or 26 timing).

1 smp / fuse 3 / fuse 5

Accuracy from a single sample, or from 3 or 5 samples fused.

n/a

Real runs were too short to fuse 3 or 5 samples from the same test in this evaluation.

Confusion matrix

Rows = who really typed, columns = who the system said. The diagonal is correct answers; anything off the diagonal is a mix-up. *newbie* is weakest because it enrolled only 3 samples.

🖥 Engineering the app

Main thread

The single thread that runs the window and receives key events.

Animator / tween / 60 fps

Our small animation engine: one timer ticks about 60 times per second and moves every animation a step. A *tween* is an animation between two values.

Splash screen

The opening card. While it shows, the app loads numpy and pre-computes the enrolled data, so it is ready at once.

Timing fidelity

How accurately we record key times. Animations add about 3 ms of jitter to dwell; flight is unaffected.

KEYLOGGD_NO_ANIM=1

An environment setting that turns animations off for the cleanest recording.

🖥 Explain mode: the pipeline, one phase at a time

Phase rail

The row of six steps you can click through: capture, zero-pad, FFT, features, normalise, decide.

Autoplay

Steps forward on its own every 3.2 seconds.

CVD-checked palette

Colours chosen to stay distinguishable for people with colour vision deficiency (colour blindness).

🖥 Running it

`pip install -r requirements.txt`

Installs the dependencies (just numpy).

`python -m module`

Runs a module of the project as a program.

Command-line flags

--no-splash, --data-dir, --k, --threshold, --tolerance, --no-threshold change behaviour without editing code.

From first principles

Implemented ourselves from the maths, not taken from a library.

📺 Takeaways & what's next

Adaptive templates

Slowly updating a user's profile as their typing changes over weeks.

Continuous mode

Checking the typist quietly in the background after login.

Encrypted templates

Storing profiles so a leaked file is useless, and never keeping raw keystroke logs.

Liveness & mimicry tests

Checking resistance to replayed recordings and to people deliberately imitating someone.

4 Numbers to remember

Real captures (6 typists, 48 samples, 1469 keystrokes)

Identification, LOOCV k -NN ($k = 3$)	79.2 % (38 / 48)
Chance level	17 %
Verification EER	19.0 %
EER threshold / operating threshold τ	1.48 / 1.56
Mean genuine / impostor distance	1.04 / 2.48
Rollover across all keys	16.0 %
Old spectral vector on the same data	56.2 %, EER 29.8 %

Synthetic benchmark (8 users $\times$ 6 pangrams $\times$ 5 = 240 samples)

Timing vector: 1 sample / 3 fused / 5 fused	92.5 / 97.9 / 100 %
Timing vector EER	5.89 %
Spectral vector: 1 / 3 / 5 fused, EER	68.8 / 72.9 / 72.9 %, 18.72 %
Per-user spread in the distance (EER)	7.98 % $\rightarrow$ 5.45 %
Tolerance $\times 1.05$: genuine rejected / impostor accepted	5.0 % / 7.3 %

Design constants

Keystrokes per sample / minimum kept	32 / 16
Features per sample	26
FFT length / low-pass cutoff	32 / bin 5 (35 %)
asinh scale	0.05 s
Spread floor	25 % of population spread
k in k -NN	3
Accept tolerance	1.05× the EER threshold
Confidence: high / moderate	margin > 1.0 / > 0.4
Samples for a stable template	12 (≈ 3 tests)

5 Questions you are likely to get

“Isn’t this just a keylogger?”

No. A keylogger steals *what* you type. We use only timing, the features never read the text, and the app runs only when you open it. We do admit that raw logs are stored as plain JSON today; encrypting templates and dropping raw logs is on the roadmap.

“Can someone copy my rhythm?”

Watching is not enough; timing is in milliseconds. A trained mimic or a replayed recording is a real risk and we have not tested it yet: it is listed as a limitation.

“Why not deep learning?”

With 48 real samples, a neural network would memorise rather than learn. Simple, explainable statistics with nearest-neighbour matching are the right tool at this scale, and every step can be shown in Explain mode.

“Why is real data worse than the benchmark?”

Real data has one session per person, very few samples for some users (newbie has 3), and free text that is different every time. The benchmark has 30 samples per user across fixed sentences.

“What if I’m tired or on another keyboard?”

Your absolute speed changes, which is why we use ratios and robust statistics. But we have not measured cross-day or cross-keyboard performance yet; that is the first thing on the “what’s next” list.

“Why only 6 people?”

A time-boxed student project. We built a synthetic benchmark precisely so accuracy claims do not rest on six people alone.

“Why did one participant disappear?”

One enrolled file turned out to contain another participant’s samples, so it was excluded and every real-data number was recomputed without it.

“How accurate is it, in one sentence?”

On real free-text typing, it names the right person out of six about 4 times in 5 from a single 32-key sample; on the benchmark, it is 100 % when a whole test’s samples are combined.